

CHRONO-SHIELD-ONION

Warrant Canary criptográfico y mesh distribuido
para infraestructuras críticas

Un enfoque de doble canal para verificación continua de integridad, detección de patrones coordinados de ataque y resiliencia operacional en redes OT/IoT.

Chrono Shield Networks (CSN)
Chrono Future Institute — División de Ciberseguridad
Bogotá, Colombia
Versión 1.0 — Octubre 2026

Resumen ejecutivo

Las infraestructuras críticas que dependen de tecnología operacional (OT) —clínicas, pequeñas plantas industriales, redes de distribución, sitios remotos con conectividad limitada— suelen operar con equipos de red convencionales que no ofrecen ninguna forma de verificar, desde fuera, si su configuración ha sido alterada o si están operando bajo coacción. Chrono-Shield-Onion es una arquitectura de referencia, de código abierto, que combina tres mecanismos complementarios: (1) un canal de administración separado del canal de datos entre nodos, (2) detección de anomalías basada en comportamiento —sin depender de firmas de ataque específicas— con correlación temporal entre nodos para distinguir fallas aisladas de patrones coordinados, y (3) un *warrant canary* criptográfico que cada nodo firma de forma autónoma y periódica, permitiendo a cualquier tercero verificar de forma independiente, sin credenciales ni acceso privilegiado, que un nodo sigue operando con su configuración original. Este documento describe la arquitectura, el razonamiento de diseño detrás de cada componente, un caso de despliegue en una instalación con varios routers distribuidos, y los resultados de las pruebas de resiliencia realizadas hasta la fecha. El objetivo es ofrecer un patrón reproducible —no un producto cerrado— que equipos de ciberseguridad OT puedan auditar, adaptar y mejorar.

Palabras clave	warrant canary, mesh mTLS, OT/ICS, detección de anomalías, integridad de configuración, resiliencia operacional
Repositorio	github.com/contactochronoshield-cyber/Chrono-Shield-Onion
Licencia	Código abierto (ver repositorio para términos específicos)
Audiencia	Equipos de seguridad OT/ICS, administradores de redes críticas, investigadores en resiliencia de infraestructura

Contenido

- 1. Introducción: el problema de confianza en redes OT
- 2. Arquitectura de doble canal
- 3. Detección de anomalías por comportamiento
- 4. Warrant Canary criptográfico
- 5. Resiliencia operacional
- 6. Caso de despliegue: instalación con routers distribuidos
- 7. Comparación con el estado del arte comercial
- 8. Limitaciones y trabajo futuro
- 9. Conclusión
- Apéndice A — Especificación de endpoints
- Apéndice B — Glosario

1. Introducción: el problema de confianza en redes OT

La mayoría de los equipos de red desplegados en infraestructura crítica —desde el router de una clínica rural hasta el gateway de un sitio industrial remoto— se diseñaron bajo un supuesto implícito: que quien tiene acceso físico o administrativo legítimo al equipo seguirá teniéndolo, y que ese acceso no estará comprometido. Ese supuesto falla en escenarios reales: robo de credenciales, coacción física sobre el personal, acceso de un proveedor de servicios que decide (voluntaria o involuntariamente) alterar la configuración, o simplemente un atacante que logra persistencia silenciosa en un dispositivo que nadie audita activamente.

El mercado de equipos de red para pequeñas y medianas instalaciones críticas —routers gestionados, gateways industriales, CPEs de seguridad— generalmente ofrece telemetría de estado (¿está encendido?, ¿responde?) pero no una forma de verificar, desde fuera y sin depender de la palabra del operador, si la configuración sigue siendo la que se desplegó originalmente. Tampoco suele distinguir entre una falla aislada (un router se cae por una sobrecarga local) y un patrón coordinado que afecta a varios nodos en una ventana de tiempo corta, que es la señal más fuerte de un incidente deliberado en lugar de un fallo operacional ordinario.

Chrono-Shield-Onion nace de un ejercicio de diseño deliberadamente honesto: en lugar de prometer *detección de ataques de inteligencia artificial* —una categoría que no deja una firma de red distinguible y que, por lo tanto, no es una promesa técnicamente sostenible— el sistema se enfoca en detección de comportamiento anómalo, que funciona de forma equivalente sin importar si el origen del ataque es un humano, un script o un modelo generativo. La pregunta que el sistema responde no es *¿quién o qué está atacando?*, sino *¿esto que estoy viendo es consistente con la operación normal de esta red?*

1.1 Principios de diseño

- Separación de canales: el tráfico administrativo nunca comparte el mismo canal que el tráfico de datos entre nodos, de forma que comprometer uno no implica automáticamente comprometer el otro.
- Verificabilidad externa: el mecanismo central (el warrant canary) puede ser verificado por cualquier tercero con la clave pública del nodo, sin necesitar credenciales ni acceso a la red interna.
- Detección sin firmas: ninguna de las reglas de anomalía depende de reconocer una técnica de ataque específica; todas se basan en desviaciones de línea base propias de cada nodo.
- Honestidad sobre las limitaciones: el sistema documenta explícitamente lo que no resuelve (ver Sección 8), en lugar de presentarse como una solución completa.

2. Arquitectura de doble canal

El sistema separa explícitamente dos superficies de red que, en la mayoría de productos comerciales, conviven en el mismo puerto y a menudo en el mismo proceso: la administración humana del sistema y la comunicación nodo-a-nodo. Esta separación reduce la superficie de ataque de cada canal y permite aplicar políticas de autenticación distintas y proporcionales al riesgo de cada uno.

	Canal de administración	Canal mesh
Puerto	5000	5443
Protocolo	HTTP + JWT	mTLS estricto (CERT_REQUIRED)
Autenticación	Usuario/contraseña (bcrypt) + token JWT con expiración y rate limiting	Certificado cliente firmado por CA propia de la instalación (OpenSSL)
Propósito	Login humano, consulta de estado, acciones administrativas (p. ej. modo mantenimiento)	Heartbeats entre nodos, registro de peers, publicación de anomalías y del warrant canary
Superficie expuesta	Dashboard, API REST de administración	Solo entre nodos de la misma instalación, certificado por certificado

Tabla 1. Separación de responsabilidades entre el canal de administración y el canal mesh.

2.1 Canal de administración (puerto 5000)

Expone endpoints REST para autenticación (`/api/v1/auth/login`), salud del proceso (`/health`), métricas en formato compatible con Prometheus vía `psutil` (`/metrics`) y telemetría (`/api/v1/telemetry`). Las contraseñas se almacenan con hash `bcrypt` (no reversible, con parámetros de costo configurables) y nunca en texto plano. Las sesiones se emiten como JWT de corta duración, y el endpoint de login aplica limitación de tasa para dificultar ataques de fuerza bruta.

2.2 Canal mesh (puerto 5443)

Cada nodo que se une a la red genera su propio par de claves y un certificado firmado por la autoridad de certificación (CA) local de la instalación. La comunicación entre nodos exige `CERT_REQUIRED` en ambos extremos: un nodo sin certificado válido no puede siquiera completar el *handshake* TLS, independientemente de si conoce o no las rutas de la API. Los nodos auto-registrados envían heartbeats periódicos que incluyen identificador de nodo, estado de recursos (CPU) y, opcionalmente, ubicación física declarada —útil quien administra una instalación con varios edificios o sitios— y esos datos quedan persistidos en una base SQLite local (`mesh_peers.db`), con tablas separadas para el registro de peers y para el historial de anomalías.

```
POST /mesh/heartbeat      # reporte periódico de cada nodo
GET  /mesh/peers          # estado actual de todos los nodos registrados
GET  /mesh/anomalies      # historial de anomalías detectadas
GET  /mesh/canary         # última declaración firmada + clave pública
POST /mesh/maintenance    # pausa alertas de integridad (requiere JWT admin)
```

Figura 1. Endpoints principales del canal mesh (puerto 5443, mTLS).

2.3 Por qué separar los canales

La separación no es solo organizativa: tiene una consecuencia de seguridad concreta. Si el canal de administración se ve comprometido —por ejemplo, por una contraseña débil o reutilizada— el atacante obtiene visibilidad y control administrativo, pero no puede hacerse pasar por un nodo ante el resto de la malla sin además poseer un certificado cliente válido, que requiere haber pasado por el proceso de emisión de la CA local. De forma simétrica, si un atacante logra comprometer un nodo individual y obtener su certificado, ese compromiso no le da automáticamente acceso a la cuenta de administración del sistema. En ambos casos, el radio de impacto de un solo punto de falla queda acotado.

3. Detección de anomalías por comportamiento

Un objetivo de diseño central del proyecto fue evitar la promesa —común en material de marketing de ciberseguridad— de *detección de ataques de IA* o de técnicas específicas de un adversario. Ningún tráfico de red deja una firma distinguible de *"esto lo generó un modelo de lenguaje"* frente a *"esto lo escribió una persona"*; cualquier producto que afirme lo contrario está vendiendo una categoría de detección que no existe de forma confiable. En su lugar, el sistema corre cuatro vigilancias independientes, cada una operando sobre desviaciones de comportamiento esperado, y las combina para distinguir ruido operacional de patrones que merecen atención humana.

Vigilancia	Condición de disparo	Clasificación emitida
Disponibilidad (offline_watcher)	Un nodo no envía heartbeat en > 30 segundos	PEER_OFFLINE
Carga de CPU	CPU reportada en el heartbeat > 90%	CPU_HIGH
Integridad de configuración (config_watcher)	El checksum SHA-256 del archivo de configuración difiere de la línea base registrada	CONFIG_TAMPERED
Correlación temporal (correlation_watcher)	El mismo tipo de anomalía ocurre en ≥ 2 nodos dentro de una ventana de 120 segundos	COORDINATED_PATTERN

Tabla 2. Las cuatro vigilancias de anomalías y sus condiciones de disparo, corriendo como hilos daemon independientes dentro de mesh_server.

3.1 El valor de la correlación temporal

La vigilancia de disponibilidad o de CPU, tomadas de forma aislada, producen muchos falsos positivos: un router se reinicia por mantenimiento, un pico de CPU ocurre por una tarea programada legítima. El valor diferencial está en correlation_watcher: cuando el mismo tipo de evento ocurre en dos o más nodos distintos dentro de una ventana corta (120 segundos en la configuración de referencia), la probabilidad de que se trate de una coincidencia operacional baja considerablemente, y el sistema lo reclasifica como COORDINATED_PATTERN, una señal que amerita revisión humana prioritaria frente a una anomalía aislada.

3.2 Integridad de configuración vía checksums

Cada nodo mantiene una línea base de checksums SHA-256 de sus archivos de configuración críticos, generada en el momento del despliegue. El config_watcher recalcula estos checksums periódicamente y los compara contra la línea base almacenada localmente (nunca en el mismo canal que se audita). Cualquier diferencia —incluida una edición legítima no documentada— dispara CONFIG_TAMPERED. Para permitir trabajo de mantenimiento legítimo sin generar alertas de falso positivo en cascada, el sistema incorpora un modo de mantenimiento con ventana de tiempo acotada (ver Sección 5.2), que pausa específicamente esta vigilancia sin desactivar las demás.

3.3 Despacho de alertas

Las anomalías detectadas se envían a través de un despachador desacoplado del motor de detección (`alert_dispatcher`), que soporta Telegram como canal opcional y degrada de forma segura a registro local (log) cuando no hay credenciales de notificación configuradas, en lugar de fallar silenciosamente o bloquear la detección.

4. Warrant Canary criptográfico

El concepto de *warrant canary* proviene del derecho: una organización publica periódicamente una declaración ("no hemos recibido ninguna orden secreta de entrega de datos"); el día que la declaración deja de publicarse, su ausencia comunica información que una orden de silencio legal impediría comunicar directamente. Chrono-Shield-Onion adapta esta idea a un contexto técnico de integridad continua: en lugar de una declaración legal, cada nodo publica periódicamente una afirmación criptográficamente firmada de que su configuración y su proceso siguen siendo los originales, verificable por cualquiera sin necesidad de confiar en la palabra del operador de la red.

4.1 Mecanismo

- Cada nodo posee un par de claves RSA-2048 generado localmente; la clave privada nunca sale del nodo ni se incluye en el control de versiones.
- Cada 60 segundos, el nodo construye una declaración de integridad y la firma con SHA-256 sobre RSA-2048, usando OpenSSL como motor criptográfico.
- La declaración firmada y la clave pública correspondiente se publican en GET /mesh/canary, accesible dentro del canal mesh.
- Cualquier verificador —interno o externo a la organización— puede comprobar la firma de forma completamente independiente, sin credenciales de la red, usando únicamente la clave pública publicada y herramientas estándar de OpenSSL.

```
# Verificación independiente, sin acceso privilegiado a la red:
openssl dgst -sha256 -verify canary_public.pem \
    -signature canary.sig canary_statement.json
# -> Verified OK
```

Figura 2. Verificación de la firma del warrant canary con herramientas estándar, sin dependencias del sistema Chrono-Shield-Onion.

4.2 Qué comunica la ausencia de la señal

El valor del mecanismo no está únicamente en la presencia de una firma válida, sino en la consecuencia de su ausencia o de un cambio no explicado en su contenido. Si un nodo deja de publicar declaraciones firmadas, o si dos declaraciones consecutivas difieren de una forma que no corresponde a una ventana de mantenimiento declarada, un verificador externo —sin necesidad de contactar al operador— puede concluir que algo en la cadena de confianza del nodo cambió. Esto traslada parte de la carga de la verificación de integridad desde "confiar en que el operador avisará si algo pasa" hacia "verificar de forma continua y automatizada, sin depender de que alguien decida comunicarlo".

4.3 Limitación reconocida

Un atacante con control total y persistente del nodo —incluyendo acceso a la clave privada de firma— podría en principio seguir publicando declaraciones válidas mientras mantiene una configuración alterada en segundo plano, de la misma forma que una orden legal de silencio con cooperación total podría, en teoría, forzar la generación de canarios falsos. Ninguna solución de software resuelve ese caso límite por completo; mitigarlo requiere controles complementarios de custodia de claves (por ejemplo, módulos de seguridad de hardware) que quedan fuera del alcance de la versión de referencia descrita aquí y se señalan explícitamente como trabajo futuro (Sección 8).

5. Resiliencia operacional

Un sistema de detección y verificación de integridad es poco útil si el propio proceso que lo ejecuta puede caerse sin recuperarse. Chrono-Shield-Onion incorpora un supervisor de procesos ligero, en lugar de depender de infraestructura de orquestación pesada que no siempre está disponible en sitios remotos o en hardware de bajo costo.

5.1 Supervisor de procesos

`supervisor.py` es un proceso Python independiente que lanza y monitorea los servicios de administración y mesh mediante `subprocess.Popen`, revisando periódicamente su estado con `poll()` y reiniciando cualquier proceso que haya terminado de forma inesperada. Esta aproximación se eligió después de descartar dos alternativas: un script de vigilancia en bash que entraba en un ciclo de muerte-reinicio bajo ciertas condiciones de fallo, y `runsvdir/termux-services`, que requiere privilegios de los que no siempre se dispone en hardware de borde restringido.

5.2 Modo de mantenimiento

Las ventanas de mantenimiento legítimo —actualizaciones de configuración autorizadas, reemplazo de hardware, calibración— generan cambios de checksum que, sin un mecanismo de excepción, dispararían `CONFIG_TAMPERED` de forma indistinguible de una alteración maliciosa. El endpoint `POST /mesh/maintenance` exige un JWT de administrador válido (es decir, requiere éxito previo en el canal de administración, con su propia autenticación) y acepta una duración en minutos, con un techo máximo de 60, durante la cual específicamente la vigilancia de integridad de configuración queda en pausa —el resto de las vigilancias (disponibilidad, CPU, correlación) permanecen activas sin interrupción.

```
curl --cert nodo.crt --key nodo.key --cacert ca.crt \
  https://<nodo>:5443/mesh/maintenance -X POST \
  -H "Authorization: Bearer <JWT admin>" \
  -H "Content-Type: application/json" \
  -d '{"minutes": 5}'
```

Figura 3. Activación de una ventana de mantenimiento de 5 minutos, autenticada contra el canal de administración.

5.3 Resultados de las pruebas de resiliencia

Prueba	Condición	Resultado observado
Caída forzada del proceso API (kill -9)	Terminación abrupta del proceso de administración	Reinicio automático por el supervisor en menos de 15 segundos
Caída forzada del proceso mesh (kill -9)	Terminación abrupta del proceso de canal mesh	Reinicio automático por el supervisor en menos de 15 segundos
Nodo sin heartbeat	Ausencia de reporte > 30 s	PEER_OFFLINE emitido de forma consistente en el umbral configurado
Cambio de configuración fuera de ventana de mantenimiento	Checksum distinto al de línea base	CONFIG_TAMPERED emitido; alerta despachada

Cambio de configuración dentro de ventana de mantenimiento	Checksum distinto, con mantenimiento activo	Sin alerta durante la ventana; reanuda vigilancia al expirar
--	---	--

Tabla 3. Resultados de pruebas funcionales de extremo a extremo realizadas sobre la implementación de referencia. Mediciones de un solo entorno de desarrollo; no sustituyen un benchmark de carga en producción (ver Sección 8).

6. Caso de despliegue: instalación con routers distribuidos

Para ilustrar el funcionamiento conjunto de los componentes descritos, se presenta un escenario representativo: una instalación crítica de tamaño pequeño —por ejemplo, una clínica o un sitio operativo remoto— con tres routers distribuidos en distintas áreas físicas (recepción/administración, área de equipos sensibles, y un enlace de respaldo hacia el exterior). Este tipo de instalación es representativo de buena parte del mercado OT de pequeña escala: pocos equipos, sin un equipo de seguridad dedicado de tiempo completo, y consecuencias operacionales reales si la red falla o si su integridad se ve comprometida sin que nadie lo note.

Nodo	Rol	Ubicación declarada
router-01	Nodo principal / salida a internet	Recepción / administración
router-02	Segmento de equipos sensibles	Área restringida
router-03	Enlace de respaldo (failover)	Cuarto de comunicaciones

Tabla 4. Distribución de nodos en el escenario de referencia de tres routers.

6.1 Flujo operativo normal

- Cada router se auto-registra ante el nodo principal al arrancar, generando su propio certificado firmado por la CA local de la instalación y declarando su ubicación física.
- Los tres nodos envían heartbeats periódicos con su estado de CPU; el dashboard administrativo (puerto 5000) permite consultar el estado agregado sin exponer el canal mesh directamente a quien solo necesita visibilidad operativa.
- Cada nodo publica su warrant canary firmado cada 60 segundos; un auditor externo —por ejemplo, un cliente corporativo que requiere evidencia de integridad continua— puede verificar la firma sin necesitar acceso a la red interna de la instalación.

6.2 Flujo ante un incidente simulado

Si un atacante obtiene acceso físico a router-02 y modifica su configuración para redirigir tráfico, el checksum de configuración de ese nodo deja de coincidir con la línea base, disparando `CONFIG_TAMPERED`. Si, en la misma ventana de tiempo, el atacante también interrumpe router-03 para evitar el failover, la vigilancia de correlación detecta dos anomalías de nodos distintos en una ventana de 120 segundos y reclasifica el evento como `COORDINATED_PATTERN`, priorizando la alerta frente a lo que de otra forma podrían interpretarse como dos incidentes aislados y de menor urgencia. De forma independiente a esta detección activa, cualquier verificador externo que estuviera monitoreando el warrant canary de router-02 habría notado la discontinuidad en las declaraciones firmadas de ese nodo, incluso si el despachador de alertas hubiera sido deshabilitado como parte del mismo ataque.

7. Comparación con el estado del arte comercial

Fabricantes establecidos de equipos de red gestionada y gateways industriales —entre ellos Zyxel, Netgear, HPE Aruba, Nexcom, Comcast Business y Rajant— ofrecen telemetría de estado, gestión centralizada y, en algunos casos, detección de anomalías de tráfico. Ninguno de estos productos, hasta donde pudo determinarse en una revisión de su documentación pública, ofrece un mecanismo de integridad verificable por terceros sin acceso privilegiado a la plataforma de gestión del fabricante: la verificación de que *"el equipo sigue configurado como se desplegó"* depende, en la práctica, de confiar en el dashboard del proveedor.

Capacidad	Equipos de red gestionada típicos	Chrono-Shield-Onion
Telemetría de estado	Sí	Sí
Detección de anomalías de tráfico	Parcial (según fabricante)	Sí, por comportamiento, sin depender de firmas de ataque
Correlación entre nodos para distinguir patrones coordinados	Poco común en el segmento de pequeña escala	Sí (ventana de 120 s)
Verificación de integridad por un tercero sin credenciales	No disponible de forma estándar	Sí, vía warrant canary criptográfico (RSA-2048/SHA-256)
Separación física de canal administrativo y canal de datos	Variable	Sí, por diseño (puertos y autenticación distintos)
Código abierto / auditable	Generalmente no	Sí

Tabla 5. Comparación cualitativa basada en documentación pública de cada fabricante. No constituye una evaluación exhaustiva ni una prueba de laboratorio comparativa; se invita al lector a verificar contra la documentación vigente de cada producto.

Esta comparación no pretende afirmar que los productos comerciales mencionados son inferiores en todas sus dimensiones —muchos ofrecen soporte, garantías de hardware y escala operativa que un proyecto de referencia de código abierto no provee por sí solo—, sino señalar un vacío específico: la verificación de integridad independiente, accesible sin privilegios, como capacidad diferenciada que hoy no es estándar en ese segmento del mercado.

8. Limitaciones y trabajo futuro

En línea con el principio de honestidad sobre las limitaciones enunciado en la Sección 1.1, esta sección documenta explícitamente lo que la versión de referencia actual no resuelve.

8.1 Limitaciones conocidas

- Custodia de claves: la clave privada de firma del warrant canary reside en el propio nodo; un compromiso total y persistente del nodo con exfiltración de esa clave permitiría, en teoría, seguir firmando declaraciones válidas sobre una configuración alterada. Mitigarlo de forma robusta requiere separar la autoridad de firma del nodo operativo, idealmente con hardware dedicado.
- Rotación de la autoridad de certificación: la rotación de la CA local (`rotate_ca.sh`) es hoy un proceso manual o programado por cron, no automático ni auto-auditado.
- Alcance de las pruebas: los resultados de resiliencia de la Sección 5.3 provienen de un entorno de desarrollo de un solo sitio; no sustituyen un benchmark de carga a escala ni pruebas en condiciones de red adversas (latencia alta, pérdida de paquetes).
- Observabilidad: no se integra todavía con plataformas de observabilidad de uso común (Grafana u otras); las métricas Prometheus actuales requieren un recolector externo.

8.2 Trabajo futuro planeado

- Separación física de la autoridad de certificación del nodo operativo, para reducir el radio de impacto de un compromiso de nodo sobre la emisión de certificados.
- Instrumentación con eBPF para visibilidad a nivel de kernel, complementaria a la detección actual basada en heartbeats y checksums.
- Exploración de almacenamiento inmutable (btrfs con snapshots) para configuración crítica, en hardware que lo soporte (fuera del alcance de los entornos de prueba basados en Android/Termux usados durante el desarrollo inicial).
- Integración opcional con mecanismos de detección física —por ejemplo, correlacionar anomalías de red con anomalías acústicas detectadas por sensores físicos en el mismo sitio— como una capa adicional de corroboración entre dominios distintos (red y entorno físico).
- Panel de observabilidad (Grafana) como capa opcional sobre las métricas ya expuestas.

9. Conclusión

Chrono-Shield-Onion propone un patrón arquitectónico, no una promesa de detección absoluta: separar el canal administrativo del canal entre nodos, vigilar comportamiento en lugar de firmas de ataque específicas, correlacionar eventos entre nodos para distinguir ruido de patrones coordinados, y —el componente más distintivo— permitir que la integridad de cada nodo sea verificada por cualquier tercero, de forma criptográfica e independiente, sin requerir acceso privilegiado a la red. Ninguno de estos mecanismos es, por separado, una novedad técnica radical; su combinación deliberada, aplicada a un segmento del mercado —instalaciones críticas de pequeña escala— que hoy carece de esta capacidad de forma estándar y accesible, es la contribución central de este documento.

El proyecto se publica como código abierto precisamente para que la afirmación de que *"la integridad es verificable"* no dependa de la palabra de quien lo construyó, sino de que cualquiera pueda leer el código, reproducir el despliegue y verificar las firmas por sí mismo. Se invita a equipos de seguridad OT/ICS, investigadores y operadores de infraestructura crítica a auditar, cuestionar y extender esta arquitectura de referencia.

Repositorio	github.com/contactochronoshield-cyber/Chrono-Shield-Onion
Organización	Chrono Shield Networks (CSN) — Chrono Future Institute
Ubicación	Bogotá, Colombia

Apéndice A — Especificación de endpoints

Canal de administración (puerto 5000, HTTP + JWT)

```
POST /api/v1/auth/login      # usuario/password -> JWT
GET  /health                 # estado del proceso
GET  /metrics                 # métricas formato Prometheus (via psutil)
GET  /api/v1/telemetry        # telemetría agregada
```

Canal mesh (puerto 5443, mTLS estricto)

```
POST /mesh/heartbeat         # node_id, cpu, location -> ack
GET  /mesh/peers              # listado de nodos y su último estado
GET  /mesh/anomalies          # historial de anomalías (tipo, nodo, timestamp)
GET  /mesh/canary             # declaración firmada + clave pública vigente
POST /mesh/maintenance        # { "minutes": N } (N <= 60), requiere JWT admin
```

Variables de entorno relevantes por nodo

```
CHRONO_NODE_ID               # identificador único del nodo
CHRONO_MAIN_NODE              # dirección del nodo principal de la malla
CHRONO_NODE_LOCATION          # ubicación física declarada (opcional)
CHRONO_JWT_SECRET             # secreto de firma de JWT (canal administrativo)
CHRONO_ADMIN_PASS_HASH        # hash scrypt de la contraseña administrativa
```

Apéndice B — Glosario

Warrant canary. Declaración periódica cuya ausencia o alteración comunica información que no podría comunicarse de forma directa; adaptada aquí a un contexto de integridad técnica continua, firmada criptográficamente.

mTLS (mutual TLS). Variante de TLS en la que ambos extremos de la conexión presentan y validan un certificado, no solo el servidor.

OT (Operational Technology). Tecnología de hardware y software dedicada a monitorear o controlar procesos físicos y dispositivos en infraestructura industrial o crítica, distinta de la TI administrativa convencional.

CA (Certificate Authority). Autoridad que firma certificados digitales, estableciendo una cadena de confianza verificable entre los nodos de una red.

Correlación temporal. Técnica de detección que agrupa eventos de distintos orígenes ocurridos dentro de una ventana de tiempo acotada para distinguir coincidencias aisladas de patrones coordinados.

Línea base de integridad. Conjunto de checksums criptográficos calculados sobre archivos de configuración en el momento de un despliegue considerado confiable, usado como referencia para detectar alteraciones posteriores.